Comprehensive Python OOP Mastery

Integrated Problem Set Testing All 4 Pillars: Abstraction, Encapsulation, Inheritance, and Polymorphism

SYSTEM DESIGN | ABSTRACT BASE CLASSES | METHOD OVERRIDING | ENCAPSULATED STATE

ABSTRACTION	ENCAPSULATION	INHERITANCE	POLYMORPHISM
ABC & @abstractmethod	Private/Protected & Getters	Base & Derived Hierarchy	Dynamic Method Dispatch

Problem 1: E-Commerce Payment & Order Processing Engine

HARD • 100 POINTS

PROBLEM STATEMENT

Design and implement a robust e-commerce payment and order processing engine that unifies diverse payment gateways under a common interface. The engine must support credit card transactions and PayPal checkouts while strictly validating sensitive credentials and managing order transaction lifecycles.

ARCHITECTURE REQUIREMENTS

1. **Abstraction (`PaymentMethod`):** Create an Abstract Base Class (ABC) named `PaymentMethod` containing an abstract method `process_payment(amount: float) -> bool` and a concrete helper method `get_status() -> str` which tracks payment state (`"PENDING"`, `"SUCCESS"`, or `"FAILED"`).
2. **Encapsulation & Polymorphism (`CreditCardPayment`):** Create a subclass of `PaymentMethod` with private attributes `__card_number` (string) and `__cvv` (string). Use `@property` setters to validate that `card_number` consists of exactly 16 digits and `cvv` consists of exactly 3 digits. Implement `process_payment(amount)`: if credentials are valid and amount > 0, set status to `"SUCCESS"` and return `True`; otherwise set status to `"FAILED"` and return `False`.
3. **Encapsulation & Polymorphism (`PayPalPayment`):** Create a subclass of `PaymentMethod` with private attribute `__email`. Validate that the email contains both `"@"` and `"."`. Implement `process_payment(amount)` verifying email validity and amount > 0.
4. **Inheritance (`Transaction` & `Order`):** Create a base class `Transaction` with attributes `transaction_id` and `timestamp`. Create a derived class `Order(Transaction)` that takes `order_id`, `item_total`, and a `PaymentMethod` object via dependency injection. Implement `checkout()` which delegates payment processing to the injected `PaymentMethod` polymorphically.

Problem 1 Specifications & Test Cases

IO SPECIFICATION

INPUT FORMAT

The input consists of execution commands simulating order processing:

- Line 1: Payment Type (`"CREDIT_CARD"` or `"PAYPAL"`)
- Line 2: Payment Credentials (Comma-separated: `card_number,cvv` OR `email`)
- Line 3: Order ID and Amount (Comma-separated: `order_id,amount`)

OUTPUT FORMAT

Print execution logs:

- Validation errors if credentials fail (e.g., `"Invalid Card Details"` or `"Invalid PayPal Email"`).
- Processing output indicating order checkout outcome and current payment status (e.g., `"Order [order_id] Processed: SUCCESS"`).

SAMPLE INPUT 1

```
CREDIT_CARD 1234567812345678,456
ORD-9901,250.75
```

SAMPLE INPUT 2

```
PAYPAL user_at_domain.com ORD-
9902,120.00
```

SAMPLE OUTPUT 1

```
Payment Status: PENDING Processing
Credit Card Payment of $250.75... Order ORD-
9901 Processed: SUCCESS Payment Status:
SUCCESS
```

SAMPLE OUTPUT 2

```
Invalid PayPal Email Order ORD-9902
Processed: FAILED Payment Status: FAILED
```

Problem 2: Automated Banking & Loan Approval System

HARD • 100 POINTS

PROBLEM STATEMENT

Build an enterprise automated banking and loan approval workflow engine that manages various bank accounts, interest calculations, and automated credit scoring for personal loan applicants.

ARCHITECTURE REQUIREMENTS

1. **Abstraction & Encapsulation (`BankAccount`):** Create an ABC named `BankAccount` with protected attribute `_account_number` and private attribute `__balance`. Provide getters/setters for `balance` that prevent setting negative balances. Define abstract method `calculate_interest() -> float` and concrete methods `deposit(amount)` and `withdraw(amount)`.
2. **Inheritance & Polymorphism (`SavingsAccount` & `CurrentAccount`):**
   - `SavingsAccount` inherits from `BankAccount` with attribute `interest_rate`. Overrides `calculate_interest()` returning `balance * interest_rate`.
   - `CurrentAccount` inherits from `BankAccount` with attribute `overdraft_limit`. Overrides `calculate_interest()` returning `0.0` and overrides `withdraw(amount)` to allow drawing up to `balance + overdraft_limit`.

Problem 2 Architecture (Cont.) & Test Cases

IO SPECIFICATION

ARCHITECTURE REQUIREMENTS (CONTINUED)

3. **Abstract Interface (`LoanApplicant`):** Create an ABC `LoanApplicant` with abstract method `evaluate_credit_risk() -> bool`.
4. **Integrated System (`PersonalLoan`):** Create a class `PersonalLoan(LoanApplicant)` initializing with `applicant_name`, an instance of `BankAccount`, requested `loan_amount`, and `credit_score` (integer 300 to 850). Implement `evaluate_credit_risk()`: return `True` if `credit_score >= 700` AND `bank_account.balance >= loan_amount * 0.20`; otherwise return `False`. Implement `approve_loan()` which prints decision logs and deposits loan funds if approved.

INPUT FORMAT

Input sequence simulating account setup and loan evaluation:

- Line 1: Account Type (`"SAVINGS"` or `"CURRENT"`), Account Number, Initial Balance, and Rate/Overdraft
- Line 2: Applicant Name, Requested Loan Amount, and Credit Score

OUTPUT FORMAT

Print execution logs:

- Calculated annual interest for the account.
- Credit evaluation result (`"APPROVED"` or `"REJECTED"`).
- Updated account balance post loan processing.

SAMPLE INPUT 1

```
SAVINGS,SA-1092,5000.0,0.04 John Doe,10000.0,740
```

SAMPLE INPUT 2

```
CURRENT,CA-5541,1000.0,2000.0 Jane Smith,15000.0,650
```

SAMPLE OUTPUT 1

```
Account SA-1092 Interest: $200.00
Evaluating Loan for John Doe... Credit Risk Assessment: PASSED Loan APPROVED! Deposited $10000.0 Updated Balance: $15000.00
```

SAMPLE OUTPUT 2

```
Account CA-5541 Interest: $0.00
Evaluating Loan for Jane Smith... Credit Risk Assessment: FAILED Loan REJECTED!
Insufficient Credit/Deposit Updated Balance: $1000.00
```

Submission Guidelines

- Use Python's `abc` module (`from abc import ABC, abstractmethod`) for abstract classes.
- Ensure proper name mangling (`__attribute`) for private encapsulation and validation checks in property setters.
- Do NOT write top-level loose code; structure all business logic strictly inside the specified class hierarchy.